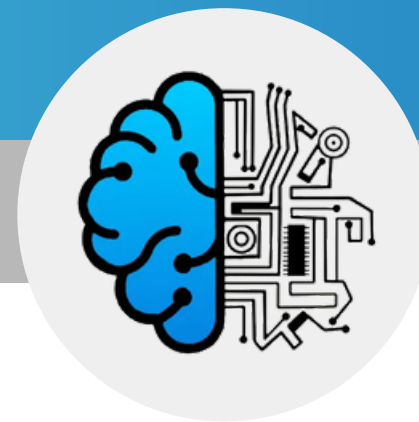




MINICURSO DE ALGORITMOS 2

PET-SIMC

RECURSIVIDADE

PET de Sistemas de Informação do Campus Monte Carmelo

@petsimc



PET de Sistemas de Informação do Campus Monte Carmelo

@petsimc

CRONOGRAMA:

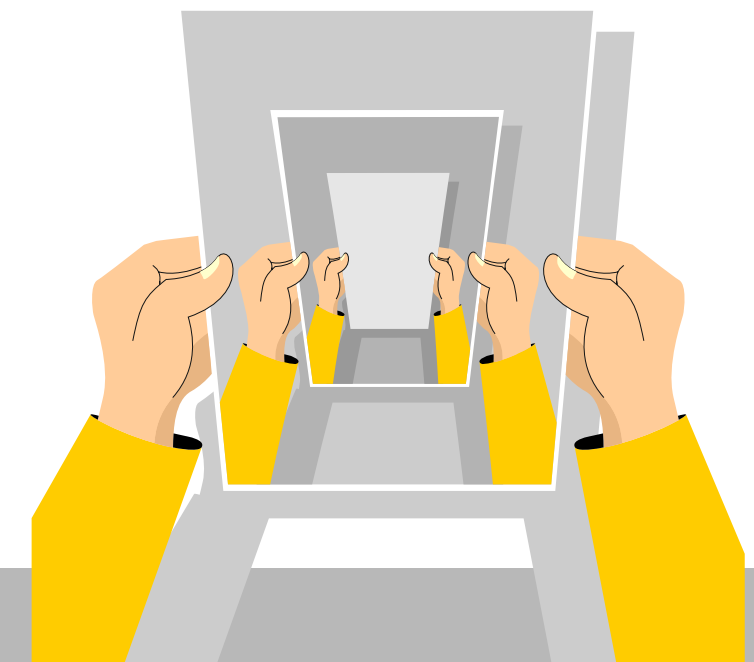
1. Conceitos
2. Sintaxe
3. Exercícios



Recursividade

Recursividade é uma **ideia inteligente** que desempenha um papel central na **programação funcional** e na ciência da computação em geral.

Para funções, **a recursão ocorre quando uma função é definida em termos de si mesma**, ou seja, chama a si própria.



@petsimc

Mas como definir essa função?

De modo geral, uma **definição de função recursiva** é dividida em duas partes:

- Há **um ou mais casos base** que dizem o que fazer em situações simples, onde não é necessária nenhuma recursão. Nestes casos **a resposta pode ser dada de imediato**, sem chamar recursivamente a função sendo definida. Isso garante que a recursão eventualmente possa parar.
- Há **um ou mais casos recursivos** que são mais gerais, **e definem a função em termos de uma chamada mais simples a si mesma**.

Fatorial

Considere a definição da função fatorial:

$$\begin{aligned} n! &= 1 \text{ se } n \leq 0 \\ & n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 3 \cdot 2 \cdot 1 \text{ se } n > 0 \end{aligned}$$

E como definição equivalente:

$$\begin{aligned} n! &= 1 \text{ se } n \leq 0 \\ & n \cdot (n-1)! \text{ se } n > 0 \end{aligned}$$

n!

Fatorial

Dizemos que essa última definição é uma definição recursiva, pois usa a função fatorial para definir a função fatorial. Chamando a função de f , temos:

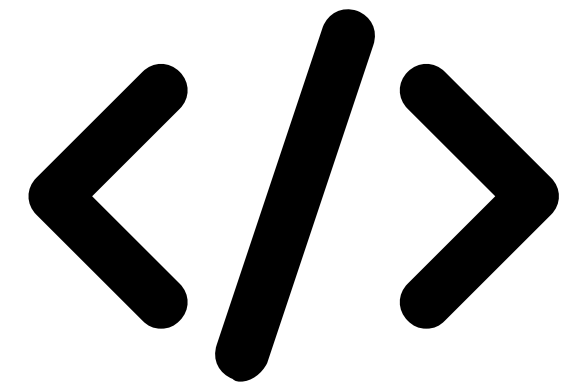
$$\begin{aligned} f(n) &= 1 \text{ se } n \leq 0 \\ & n \cdot f(n-1) \text{ se } n > 0 \end{aligned}$$

$n!$

Como fica no código?

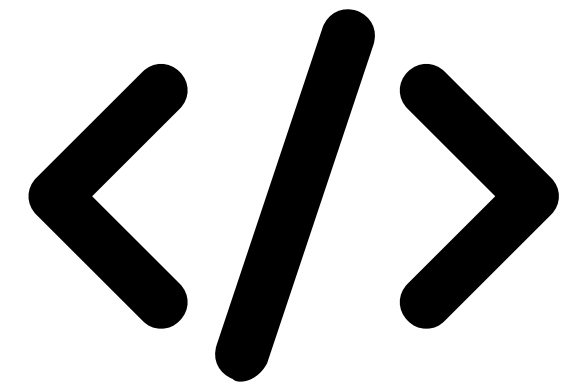
```
#include <stdio.h>
#include <stdlib.h>

int fatrec(int n) {
    int f;
    printf("\nchamando fatorial de %10d", n);
    if (n <= 0) return 1;
    f = n * fatrec(n-1);
    printf("\nfeita a multiplicacao por %10d", n);
    return f;
}
```



Como fica no código?

```
int main() {  
    int n; /* numero dado */  
    /* ler o n */  
    printf("digite o valor de n:");  
    scanf("%d", &n);  
    /* calcule e imprima o resultado de fatorial de n */  
    printf("\n\nfatorial de %10d = %10d\n", n, fatrec(n));  
    system("PAUSE");  
    return 0;  
}
```



Uma ilustração do fatorial de 5:

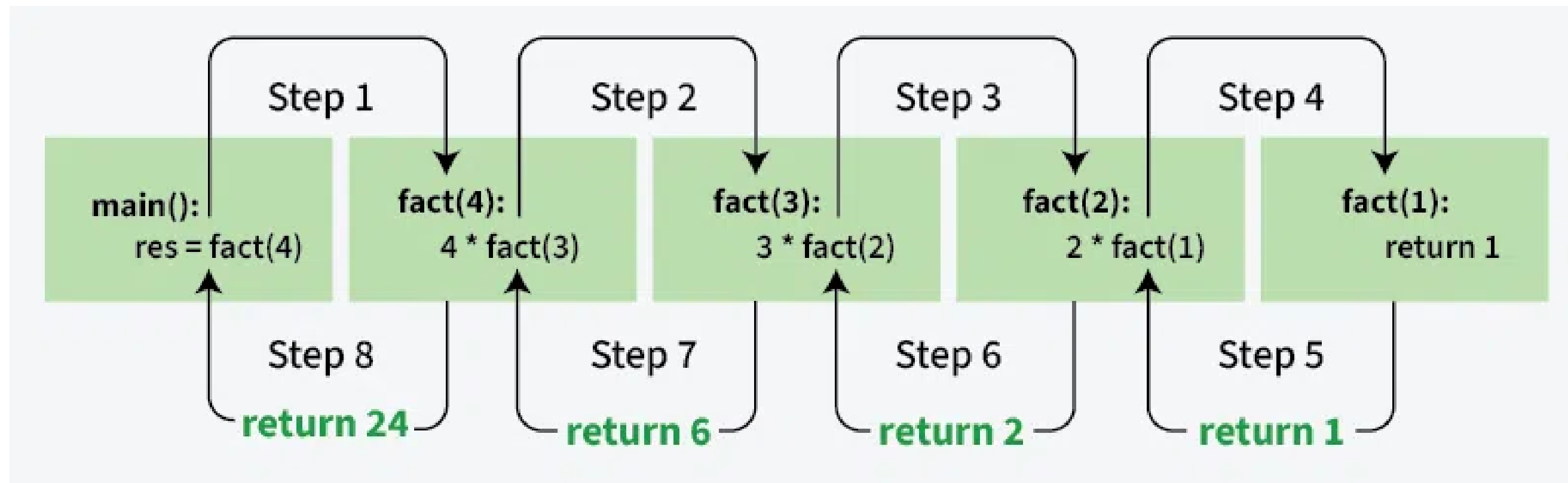


Imagem retirada do site geeksforgeeks

Pontos críticos

- É preciso um cuidado especial para não cairmos em um looping infinito.
- **Ponto de Parada ou Condição de Parada:** é o ponto onde a função será encerrada.
- Toda função recursiva precisa de uma condição de parada.

- É preciso um cuidado especial para não cairmos em um looping infinito.
- **Ponto de Parada ou Condição de Parada:** é o ponto onde a função será encerrada.
- Toda função recursiva precisa de uma condição de parada.

Vamos praticar?!

@petsimc



Exercício 01

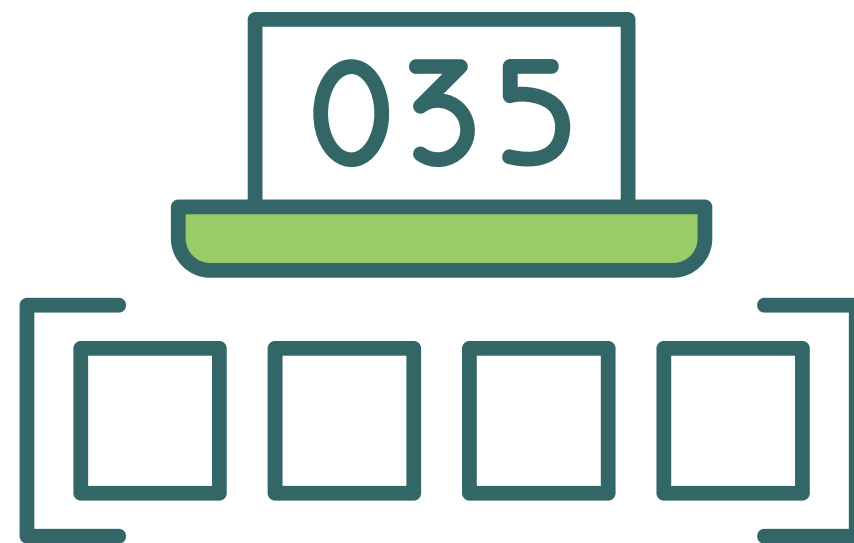
Faça uma função recursiva que calcule e retorne o fatorial de um número inteiro N.

$n!$

@petsimc

Exercício 02

Faça uma função recursiva que permita somar os elementos de um vetor de inteiros.



Exercício 03

Crie uma função recursiva que receba um número inteiro positivo N e calcule o somatório dos números de 1 a N .



@petsimc

Referências

<http://www.decom.ufop.br/romildo/2012-1/bcc222/slides/06-recursividade.pdf>

<https://www.ime.usp.br/~mms/mac1222s2014/12%20-%20algoritmos%20e%20funcoes%20recursivas.pdf>

CONTATO

Como entrar em contato em caso de dúvidas?



E-mail

laviniabd@ufu.br
nicolly.luz@ufu.br



Sala do PET- 1BMC206

UFU - Monte
Carmelo - Campus
Araras



Site do PET-SIMC

petsimc.facom.ufu.br



Grupo do Wpp



@petsimc